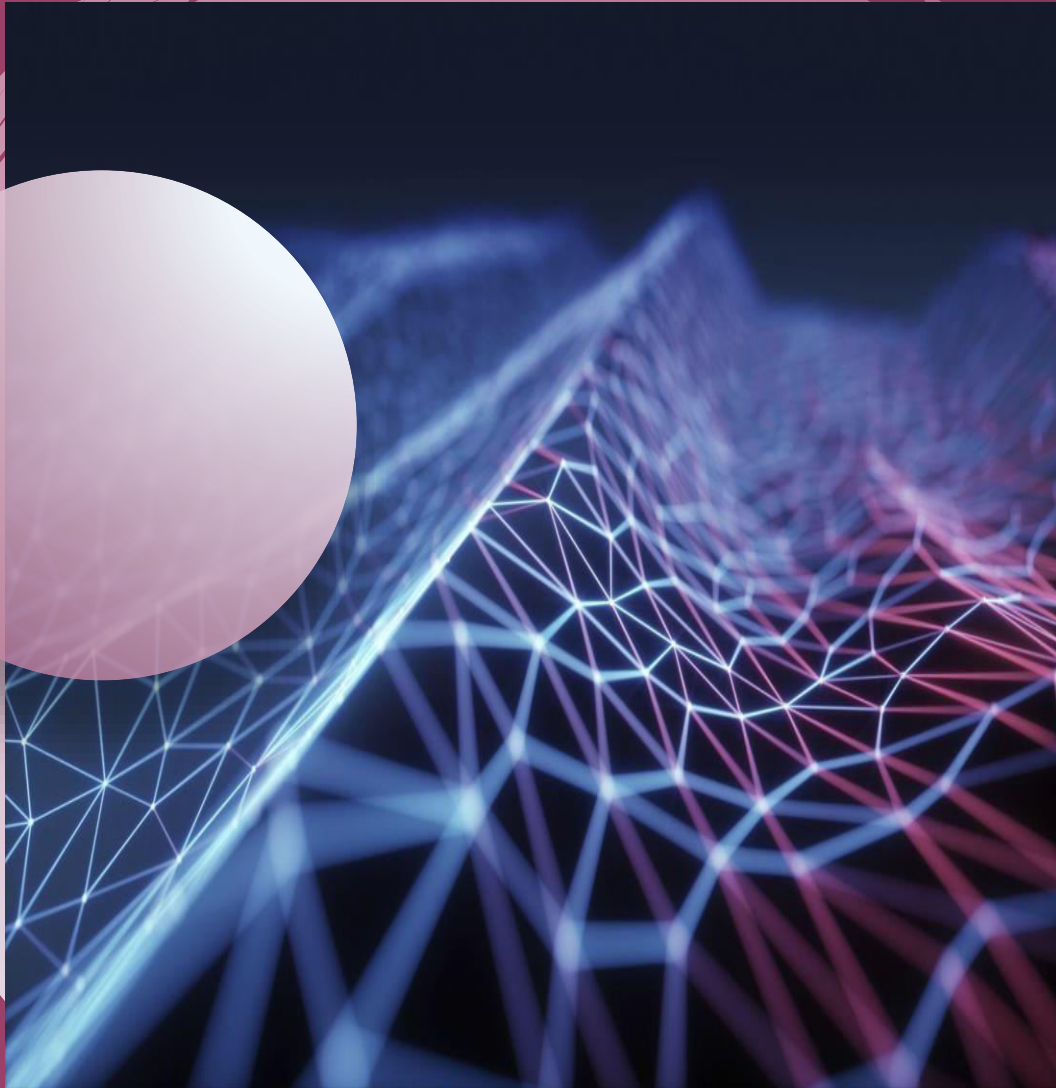


# WELCOME

Tajwar Mehmood  
Maryam Shahbaz  
Bushra Fatima

[tajwar.mehmood@nu.edu.pk](mailto:tajwar.mehmood@nu.edu.pk)  
[maryam.shahbaz@nu.edu.pk](mailto:maryam.shahbaz@nu.edu.pk)  
[bushra.fatima@nu.edu.pk](mailto:bushra.fatima@nu.edu.pk)

GCR CODE : kvjwm7xq



# THEORY OF AUTOMATA

WEEK 1

# Text and Reference Material

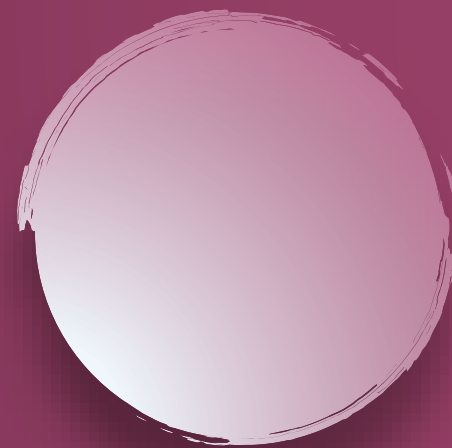
1. *Introduction to Computer Theory*, by Daniel I. Cohen, John Wiley and Sons, Inc., 1991, Second Edition
2. *Introduction to Languages and Theory of Computation*, by J. C. Martin, McGraw Hill Book Co., 1997, Second Edition

# Contents

- Introduction
  - Automata
  - languages
- Basics
  - Alphabet
  - Strings, Words, Concatenation, Length
  - Grammar
  - Language

# Contents

- Kleene Closure
  - Lexicographic order
  - Definition
  - Kleene Closure of null set & set with only null word
  - Kleene Closure of different sets
  - Positive Kleene Closure
  - $S^{**}$



# INTRODUCTION

# What does automata mean?

- It is the plural of automaton, and it means “something that works automatically”



# Theory of Automata

Theory of automata is a theoretical branch of computer science and mathematical. It is the study of abstract machines and the computation problems that can be solved using these machines.

## Automata

The word *automata* comes from the Greek word αὐτόματος, which means "self-acting, self-computing, self-moving".

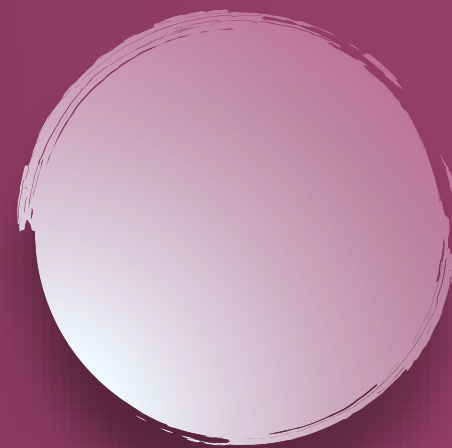
It is the plural of automaton, and it means "something that works automatically"

# Introduction to languages

- There are two types of languages
  - Formal Languages (Syntactic languages)
  - Informal Languages (Semantic languages)

# Few Applications of Theory of Automata

1. Compiler Design : Syntax analysis uses context-free grammars and pushdown automata for parsing
2. Robotics: Finite State Machines (FSMs) are used to model and control the behavior of robots, defining their actions based on different states and inputs.
3. DNA Sequencing : Automata can be used to analyze and compare DNA sequences.
4. Text Processing and Search: Pattern matching (e.g., in text editors, search engines, plagiarism detection).



BASICS

# The Central Concepts of Automata Theory

- In this Lecture we shall introduce the most important definition of terms that pervade the theory of automata.
- These concepts includes
  - Alphabet
  - Strings
  - Grammar
  - Language

# Alphabets

- An alphabet is a finite, nonempty set of symbols. Conventionally we use the symbol  $\Sigma$  for an alphabet
- Common alphabet includes
  - $\Sigma = \{ 0, 1 \}$ , the binary alphabet
  - $\Sigma = \{ a, b, c, d, \dots z \}$ , the set of all lower case letters.

## NOTE:

- A certain version of language ALGOL has 113 letters

$\Sigma$  (alphabet) includes letters, digits and a variety of operators including sequential operators such as GOTO and IF

# Strings

- Definition:

Concatenation of finite symbols from the alphabet is called a string.

- Example:

If  $\Sigma = \{a, b\}$  then

a, abab, aaabb, abababababababababab



## EMPTY STRING or NULL STRING:

- Sometimes a string with no symbol at all is used, denoted by (Small Greek letter Lambda)  $\lambda$  or (Capital Greek letter Lambda)  $\Lambda$ , is called an empty string or null string.

The capital lambda will mostly be used to denote the empty string, in further discussion.

# The Empty Strings

- The Empty String is the string with zero occurrences of symbols.
- The string denoted by  $\epsilon$  or  $\Lambda$  or  $\lambda$
- Is a string that may be chosen from any alphabet whatsoever.

# Words

- Definition:

Words are strings belonging to some language.

Example:

If  $\Sigma = \{x\}$  then a language  $L$  can be defined as

$L = \{x^n : n=1,2,3,\dots\}$  or  $L = \{x,xx,xxx,\dots\}$

Here  $x,xx,\dots$  are the words of  $L$

## NOTE:

- All words are strings, but not all strings are words.

# Word / String

- Consider a nonempty set  $A$  of symbols. A **word** or **string**  $w$  on the set  $A$  is a finite sequence of its elements.
- For example, suppose  $A = \{a, b, c\}$ . Then the following sequences are **words** on  $A$ :
  - $u = ababb = abab^2$
  - $v = accbaaa = ac^2ba^3$ .

# Strings

- Example # 1
  - **01101** is a string from the binary alphabet  $\Sigma \{0, 1\}$
- Example # 2
  - **hello** is a string from the alphabet  $\Sigma \{a, b, c, d, \dots z\}$

# The Length of a String

- It is often useful to classify strings by their length i.e. the number of position for symbol in the string.
- For example 01101 has length 5.
- It is common to say that the length of a string is “the number of symbols” in the string;

# The Length of a String

- The standard notation for the length of a string  $w$  is  $|w|$ .
- $|011| = 3$
- $|\epsilon| = 0$



# Length of Strings

- Definition:

The length of string  $s$ , denoted by  $|s|$ , is the number of letters in the string.

- Example:

$\Sigma = \{a, b\}$

$s = ababa$

$|s| = 5$

- Example:

$\Sigma = \{B, aB, bab, d\}$

$s = BaBbabBd$

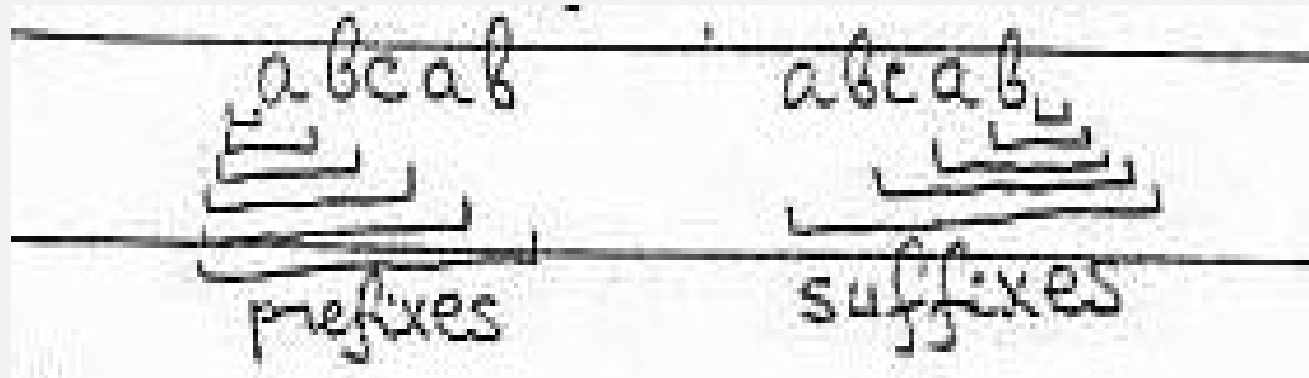
Tokenizing = (B), (aB), (bab), (d)

# Substring

- A part of a string is known as substring
- $L_1 = \text{abca}$
- **bca** is a substring of  $L_1$
- **ca** is a substring of  $L_1$

# Prefixes and Suffixes

- A beginning of a string (up to any symbol) is a prefix and an ending is a suffix



- A string is suffix and prefix of itself
- $\epsilon$  is suffix and prefix of any string

# Valid/In-valid alphabets

- While defining an alphabet, an alphabet may contain letters consisting of group of symbols for example  $\Sigma_1 = \{B, aB, bab, d\}$ .
- Now consider an alphabet  $\Sigma_2 = \{B, Ba, bab, d\}$  and a string BababB.

This string can be tokenized in two different ways

- (Ba), (bab), (B)
- (B), (abab), (B)

Which shows that the second group cannot be identified as a string, defined over

$\Sigma = \{a, b\}$ .

## Remarks

- As when this string is scanned by the compiler (Lexical Analyzer),
  - first symbol B is identified as a letter belonging to  $\Sigma$ ,
  - while for the second letter the lexical analyzer would not be able to identify,
  - so while defining an alphabet it should be kept in mind
    - that ambiguity should not be created.

## Remarks:

- While defining an alphabet of letters consisting of more than one symbols, no letter should be started with the letter of the same alphabet *i.e.* one letter should not be the prefix of another.
- However, a letter may be ended in the letter of same alphabet *i.e.* one letter may be the suffix of another.



## Conclusion

- $\Sigma_1 = \{B, aB, bab, d\}$
- $\Sigma_2 = \{B, Ba, bab, d\}$

$\Sigma_1$  is a valid alphabet while  $\Sigma_2$  is an in-valid alphabet.

## *Lexicographic ordering*

- The Lexicographic ordering of strings is the same as the dictionary ordering, except that shorter strings precede longer strings.
- The lexicographic ordering of all strings over the alphabet  $\{0, 1\}$  is  $(\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots)$ .

# The Power of an Alphabet

- If  $\Sigma$  is an alphabet we can express the set of all strings of a certain length from that alphabet by using an exponential notation.
- We define  $\Sigma^k$  to be the set of strings of length  $k$ , each of whose symbols is in  $\Sigma$ .

•

# The Power of an Alphabet

- Examples if  $\Sigma = \{0, 1\}$
- $\Sigma^0 = \{\epsilon\}$  regardless of what alphabet  $\Sigma$  is.
- $\Sigma^1 = \{0, 1\}$
- $\Sigma^2 = \{00, 01, 10, 11\}$
- $\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

# The Power of an Alphabet

- The confusion between  $\Sigma^k$  and  $\Sigma$ 
  - $\Sigma$  is an alphabet and its members 0 and 1 are symbols.
  - $\Sigma^k$  is a set of strings its members are the strings (of) 0 and 1, each of which is of length  $k$ .

# The Power of an Alphabet: Star Closure or Kleene's Star

- $\Sigma^* = ?$
- The set of all strings over an alphabet  $\Sigma$ .
- For Example  $\{0, 1\}^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$
- $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \Sigma^4 \cup \Sigma^5 \dots$

# The Power of an Alphabet: Positive Closure or Kleene's Plus

- Sometimes, we wish to exclude the empty string from the set of strings.
- The set of nonempty strings from alphabet  $\Sigma$  is denoted by  $\Sigma^+$
- Thus
- $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \Sigma^4 \cup \Sigma^5 \dots$
- $\Sigma^* = \Sigma^+ \cup \{\epsilon\}$

# Concatenation of Strings

- Let  $x$  and  $y$  be strings. Then  $xy$  denotes the concatenation of  $x$  and  $y$ .
- $xy$  = string formed by making a copy of  $x$  and following it by a copy of  $y$ .
- Precisely if  $x = a_1a_2a_3a_4...a_i$  and  $y = b_1b_2b_3b_4...b_j$
- $xy = a_1a_2a_3a_4...a_i b_1b_2b_3b_4...b_j$



# Concatenation of Strings

- Let
  - $x = 01101$
  - $y = 110$

Then

- $xy = 01101110$
- $yx = 11001101$

# Concatenation of Strings

- Remember that for any string  $w$

$$\epsilon w = w \epsilon = w$$

- The identity for concatenation

# Reverse of a String

- If  $W = abc$
- Then  $W^R = W$  in reverse order
- $W^R = cba$

# Reverse of a String

- Definition:

The reverse of a string  $s$  denoted by  $\text{Rev}(s)$  or  $s^r$ , is obtained by writing the letters of  $s$  in reverse order.

- Example:

If  $s=abc$  is a string defined over  $\Sigma=\{a,b,c\}$   
then  $\text{Rev}(s)$  or  $s^r = cba$

- Example:

$\Sigma = \{B, aB, bab, d\}$

$s = BaBbabBd$

$\text{Rev}(s) = dBbabaBB$

# Introduction to Grammar

Grammar is a mechanism to describe the languages.

A grammar ( $G$ ) is defined as a quadruple

$$G = (V, T, S, P)$$

where

$V$  = Finite set of objects called VARIABLES

$T$  = Finite set of objects called TERMINAL SYMBOLS

$S \in V$  = Start variables

$P$  = Finite set of Productions.

A production rule  $P$  is of the form

$$x \rightarrow y$$

# Language

A language is a collection of appropriate string.

## Example: 1

$L1 = \{\text{Set of string of length 2}\}$

$= \{aa, bb, ba, bb\}$

**Finite Language**

## Example: 2

$L2 = \{\text{Set of all strings starts with 'a'}\}$

$= \{a, aa, aaa, abb, abbb, ababb\}$

**Infinite Language**



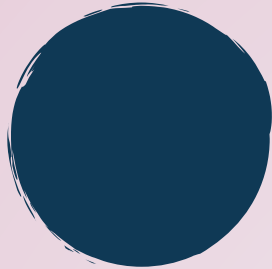
LANGUAGE CAN BE  
DEFINED BY  
FOLLOWING WAYS



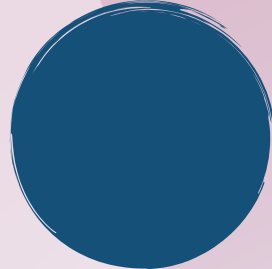
# Language can be defined by following ways:



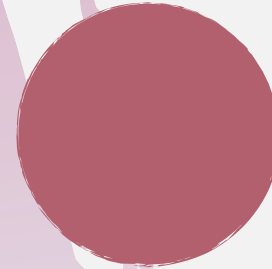
Descriptive  
Definition



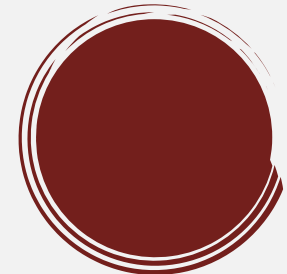
Recursive  
Refinition



Regular  
Expressions(RE)



Finite  
Automata(FA)



More



# DESCRIPTIVE DEFINITION

## **Descriptive definition of language:**

The language is defined, describing the conditions imposed on its words.

- Example:

The language  $L$  of strings of odd length, defined over  $\Sigma=\{a\}$ , can be written as

$L=\{a, aaa, aaaaa, \dots\}$

- Example:

The language  $L$  of strings that does not start with  $a$ , defined over  $\Sigma=\{a,b,c\}$ , can be written as

$L=\{b, c, ba, bb, bc, ca, cb, cc, \dots\}$

- Example:

The language  $L$  of strings of length 2, defined over  $\Sigma=\{0,1,2\}$ , can be written as

$$L=\{00, 01, 02, 10, 11, 12, 20, 21, 22\}$$

- Example:

The language  $L$  of strings ending in 0, defined over  $\Sigma=\{0,1\}$ , can be written as

$$L=\{0, 00, 10, 000, 010, 100, 110, \dots\}$$

- Example: The language **EQUAL**, of strings with number of a's equal to number of b's, defined over  $\Sigma=\{a,b\}$ , can be written as

$\{\Lambda, ab, aabb, abab, baba, abba, \dots\}$

- Example: The language **EVEN-EVEN**, of strings with even number of a's and even number of b's, defined over  $\Sigma=\{a,b\}$ , can be written as

$\{\Lambda, aa, bb, aaaa, aabb, abab, abba, baab, baba, bbaa, bbbb, \dots\}$

- Example: The language **INTEGER**, of strings defined over  $\Sigma=\{-,0,1,2,3,4,5,6,7,8,9\}$ , can be written as

$\text{INTEGER} = \{\dots,-2,-1,0,1,2,\dots\}$

- Example: The language **EVEN**, of strings defined over  $\Sigma=\{-,0,1,2,3,4,5,6,7,8,9\}$ , can be written as

$\text{EVEN} = \{\dots,-4,-2,0,2,4,\dots\}$

- Example: The language  $\{a^n b^n\}$ , of strings defined over  $\Sigma=\{a,b\}$ , as  $\{a^n b^n : n=1,2,3,\dots\}$ , can be written as  $\{ab, aabb, aaabbb, aaaabbbb, \dots\}$
- Example: The language  $\{a^n b^n a^n\}$ , of strings defined over  $\Sigma=\{a,b\}$ , as  $\{a^n b^n a^n : n=1,2,3,\dots\}$ , can be written as  $\{aba, aabbaa, aaabbbbaaa, aaaabbbbbaaaa, \dots\}$



- Example: The language **factorial**, of strings defined over  $\Sigma=\{1,2,3,4,5,6,7,8,9\}$  *i.e.*  
 $\{1,2,6,24,120,\dots\}$
- Example: The language **FACTORIAL**, of strings defined over  $\Sigma=\{a\}$ , as  
 $\{a^{n!} : n=1,2,3,\dots\}$ , can be written as  
 $\{a,aa,aaaaaa,\dots\}$ . It is to be noted that the language **FACTORIAL** can be defined over any single letter alphabet.

- Example: The language **DOUBLEFACTORIAL**, of strings defined over  $\Sigma=\{a, b\}$ , as  $\{a^{n!}b^{n!} : n=1,2,3,\dots\}$ , can be written as  $\{ab, aabb, aaaaaabbbbbbb,\dots\}$
- Example: The language **SQUARE**, of strings defined over  $\Sigma=\{a\}$ , as  $\{a^{n^2} : n=1,2,3,\dots\}$ , can be written as  $\{a, aaaa, aaaaaaaaaa,\dots\}$

- Example: The language **DOUBLESQUARE**, of strings defined over  $\Sigma=\{a,b\}$ , as  
 $\{a^{n^2} b^{n^2} : n=1,2,3,\dots\}$ , can be written as  
 $\{ab, aaaabbbb, aaaaaaaaaabbbbbbbbbbb,\dots\}$

- Example: The language **PRIME**, of strings defined over  $\Sigma=\{a\}$ , as  $\{a^p : p \text{ is prime}\}$ , can be written as  $\{aa,aaa,aaaaa,aaaaaaaa,aaaaaaaaaaaaa...\}$

## An Important language

- **PALINDROME:**

The language consisting of  $\Lambda$  and the strings  $s$  defined over  $\Sigma$  such that  $\text{Rev}(s)=s$ .

It is to be denoted that the words of PALINDROME are called palindromes.

- Example: For  $\Sigma=\{a,b\}$ ,

$\text{PALINDROME}=\{\Lambda, a, b, aa, bb, aaa, aba, bab, bbb, \dots\}$

## Remark

- There are as many palindromes of length  $2n$  as there are of length  $2n-1$ .

To prove the above remark, the following is to be noted:

## Note

- Number of strings of length 'm' defined over alphabet of 'n' letters is  $n^m$ .
- Examples:
  - The language of strings of length 2, defined over  $\Sigma=\{a,b\}$  is  $L=\{aa, ab, ba, bb\}$  *i.e.* number of strings =  $2^2$
  - The language of strings of length 3, defined over  $\Sigma=\{a,b\}$  is  $L=\{aaa, aab, aba, baa, abb, bab, bba, bbb\}$  *i.e.* number of strings =  $2^3$